DevOps Engineer — Home Assignment

Time budget: max 1 day (roughly 6–8 hours). Use your own AWS account (see the cost note near the end).

This assignment is meant to be worked on with whatever tools you normally use, AI included. What matters is that you can explain and defend every decision — including how you got there, not just what the final answer was.

Context

You've been handed a small service and its infrastructure code from a teammate who left the team. It's supposed to deploy a containerized app to ECS Fargate behind an Application Load Balancer, provisioned by Terraform, deployed by a CI/CD pipeline. None of it currently works end-to-end. Your job is to recreate the project below exactly as given, get it running, understand why it was broken, and make a few calls about how you'd productionize it — two of which you'll actually implement, not just describe.

Setup

Recreate the following file structure locally (or in a fresh git repo) and copy each file's content exactly as shown in the appendix — byte for byte, including the parts that look wrong. Don't "fix as you type"; the bugs are the point.

```
devops-assignment/
  app/
    app.py
    requirements.txt
    Dockerfile
  terraform/
    versions.tf
    variables.tf
    network.tf
    security_groups.tf
    iam.tf
    ecr.tf
    alb.tf
    ecs.tf
    outputs.tf
  .github/workflows/deploy.yml
```

Part 1 — Get it working (the majority of your time)

Fix whatever is broken so that: the pipeline can build and push the image, Terraform applies cleanly, and the ALB's public DNS name serves the app's `/` endpoint over HTTP with a healthy target in the target group — and stays healthy. A deployment that applies cleanly once and then never reaches a stable state doesn't count.

There is more than one bug, and they span more than one layer (container, IaC, networking, IAM, CI/CD, and deployment behavior). Fixing one won't be enough to get a healthy deployment — and getting a healthy target group is not by itself proof the app is reachable end-to-end from the outside. Confirm both.

> One of the bugs in this repo won't show up from reading the files. It only shows up once you actually deploy: `terraform apply` will succeed, but the service will not settle — tasks will start and then get replaced or rolled back. Reading the Terraform won't tell you why; you'll need to watch it happen and correlate ECS service events, task stop reasons, and target health timestamps to find the actual cause. Don't just change numbers until it goes green — capture the evidence that told you what was wrong, the same way you would for the others.

You do not need to make the GitHub Actions pipeline literally run (unless you want to) — reasoning through and fixing it in place is enough, as long as your written submission explains what was wrong and shows the corrected file.

Part 2 — Two things to actually build, two things to answer

Build (implement, don't just describe):

1. Secrets.

The task definition currently passes a database password as a plain environment variable. Move it to AWS Secrets Manager and wire it into the task definition properly (the ECS `secrets` block, not `environment`), including whatever IAM permission the execution role needs to actually resolve it at task startup. Show it working — a task that starts successfully with the secret injected this way.

2. Autoscaling.

A single Fargate task has no autoscaling. Add an Application Auto Scaling target and a target-tracking policy on the ECS service (your choice of metric — CPU, memory, or ALB request count per target — but justify the choice in your write-up using an actual number from your own deployment, not a generic argument), scaling between a sensible min and max task count. You don't need to actually generate load to trigger it; screenshot the configured policy and explain how you'd validate it under load if you had more time.

Answer (short answers, no code required):

3. Fargate vs. EC2 vs. EKS.

Would you have chosen ECS Fargate, ECS on EC2, or EKS for this workload if you were starting from scratch, for a small team maintaining this long-term? Justify it, grounded in something concrete from this exercise (cost, ops burden, a number you actually observed) — there's no single right answer, but there is a wrong justification.

4. Code review flag.

Name one thing in this repo you'd flag in a real code review even though it isn't strictly "broken."

Part 3 — What to submit

Put everything below into one file, `SUBMISSION.md`, at the root of your repo — don't send separate docs, emails, or attachments. Everything needed to review your work should be reachable from that one file.

- **Repo access** — push your finished repo to GitHub or GitLab, with real commit history, not a single squashed commit. Commit as you go, the same way you would on a real debugging session. Add **roman.got@allcloud.io** as a collaborator on the repo (or make it public) so it can be reviewed directly — a zip file or a link that requires a request to access isn't sufficient.
- **What was broken and why**, in your own words — for each bug, what the symptom looked like (error message / CloudWatch log / unhealthy target / connection timeout / service event) and how you traced it to the cause.
- **Proof it actually ran** — a screenshot or terminal output of `curl` against the ALB DNS name returning a 200 from outside your VPC, plus a screenshot of the healthy target in the target group. Embed the images directly in SUBMISSION.md (or link them from an `/evidence` folder in the same repo) — don't paste them into a separate doc.
- **Proof of the Secrets Manager task startup and the autoscaling policy configuration** — screenshots or CLI output, embedded the same way.
- **A short screen recording (5–10 minutes, unedited)** of you diagnosing the deployment/rollback bug described in Part 1 — the one that only shows up once you deploy. Narrate it live: what you expected, what you actually saw in the console/CLI, and how that changed your hypothesis. This should capture you finding the cause, not you explaining it after the fact. Any screen recorder is fine (Loom, QuickTime, asciinema, etc.) — link it from SUBMISSION.md.
- Answers to questions 3 and 4 from Part 2.

If it's not in SUBMISSION.md or linked from it within the same repo, assume it won't be reviewed.

> We look at commit timing and history as part of understanding your process. There's no penalty for using AI — the note below says so explicitly — but a write-up that lands in a single commit right at the end, or a document that's clearly written before the corresponding work happened, is treated as a signal worth asking about in a follow-up conversation, not as a red flag on its own.

Cost note

Fargate is not covered by the AWS Free Tier — you'll be billed per vCPU/GB-hour from the first task (a day of intermittent testing is on the order of cents to low dollars; Secrets Manager and a NAT Gateway, if you end up needing one, cost a little more but are still trivial for this scope). New AWS accounts currently get $200 in credit plus a 6-month free plan, which will comfortably cover this. Destroy your resources (`terraform destroy`) when you're done so nothing keeps running — a NAT Gateway left running is the one thing here that can actually add up.

A note on using AI

Use whatever tools you normally use, including AI assistants. Pasting this file into a model and shipping whatever it suggests, without understanding it or actually running it against your own AWS account, will be obvious from the submission itself — and from your ability to walk through your specific fixes, your specific logs, and the reasoning behind your answers in a follow-up conversation.

Appendix — file contents

Copy each block below into the file path shown, exactly as-is.

app/app.py

```
from flask import Flask, jsonify
import os
import socket
import time

app = Flask(__name__)

# NOTE: warms a local cache before accepting traffic. Not configurable
# via env var yet - hardcoded for now, revisit later.
time.sleep(25)


@app.route("/")
def root():
    return jsonify(
        message="hello from devops-assignment",
        hostname=socket.gethostname(),
        version=os.environ.get("APP_VERSION", "unknown"),
    )


@app.route("/health")
def health():
    return jsonify(status="ok"), 200


if __name__ == "__main__":
    # app listens on 8080 inside the container
    app.run(host="0.0.0.0", port=8080)
```

app/requirements.txt

```
flask==3.0.3
```

app/Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
# NOTE: app.py listens on 8080 - check this matches what's below
EXPOSE 5000
CMD ["python", "app.py"]
```

terraform/versions.tf

```
terraform {
  required_version = ">= 1.5"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
```

```
}
```

terraform/variables.tf

```
variable "aws_region" {
  type    = string
  default = "us-east-1"
}

variable "project_name" {
  type    = string
  default = "devops-assignment"
}

variable "container_port" {
  description = "Port the container listens on"
  type        = number
  default     = 5000
}

variable "app_version" {
  type    = string
  default = "1.0.0"
}

# TODO(candidate): this is not how secrets should be handled in a real
# environment. You're implementing the fix for this one, not just writing
# about it - see Part 2, item 1.
variable "db_password" {
  type    = string
  default = "SuperSecretPassword123!"
}
```

terraform/network.tf

```
data "aws_vpc" "default" {
  default = true
}

data "aws_subnets" "default" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}
```

terraform/security_groups.tf

```
resource "aws_security_group" "alb" {
  name        = "${var.project_name}-alb-sg"
  description = "Allow inbound HTTP from the internet"
  vpc_id      = data.aws_vpc.default.id

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
```

```
  }
}

resource "aws_security_group" "ecs_tasks" {
  name        = "${var.project_name}-ecs-sg"
  description = "Allow inbound from the ALB only"
  vpc_id      = data.aws_vpc.default.id

  ingress {
    description     = "From ALB"
    from_port       = 5000
    to_port         = 5000
    protocol        = "tcp"
    security_groups = [aws_security_group.alb.id]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

terraform/iam.tf

```
data "aws_iam_policy_document" "ecs_assume_role" {
  statement {
    actions = ["sts:AssumeRole"]
    principals {
      type        = "Service"
      identifiers = ["ecs-tasks.amazonaws.com"]
    }
  }
}

resource "aws_iam_role" "ecs_task_execution_role" {
  name               = "${var.project_name}-task-execution-role"
  assume_role_policy = data.aws_iam_policy_document.ecs_assume_role.json
}

# NOTE: hand-rolled instead of using the AWS managed policy.
# Double check this actually covers everything ECS needs to pull
# from ECR and ship logs to CloudWatch.
resource "aws_iam_role_policy" "ecs_task_execution_custom" {
  name = "${var.project_name}-task-execution-custom-policy"
  role = aws_iam_role.ecs_task_execution_role.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Action = [
          "ecr:BatchCheckLayerAvailability",
          "ecr:GetDownloadUrlForLayer",
          "ecr:BatchGetImage"
        ]
        Resource = "*"
      }
    ]
  })
}
```

```
resource "aws_iam_role" "ecs_task_role" {
  name               = "${var.project_name}-task-role"
  assume_role_policy = data.aws_iam_policy_document.ecs_assume_role.json
}
```

terraform/ecr.tf

```
resource "aws_ecr_repository" "app" {
  name                 = var.project_name
  image_tag_mutability = "MUTABLE"

  image_scanning_configuration {
    scan_on_push = true
  }
}
```

terraform/alb.tf

```
resource "aws_lb" "app" {
  name               = "${var.project_name}-alb"
  internal           = false
  load_balancer_type = "application"
  security_groups    = [aws_security_group.alb.id]
  subnets            = data.aws_subnets.default.ids
}

resource "aws_lb_target_group" "app" {
  name        = "${var.project_name}-tg"
  port        = 5000
  protocol    = "HTTP"
  vpc_id      = data.aws_vpc.default.id
  target_type = "ip"

  health_check {
    path                = "/healthz"
    healthy_threshold   = 2
    unhealthy_threshold = 3
    timeout             = 5
    interval            = 15
    matcher             = "200"
  }
}

resource "aws_lb_listener" "app" {
  load_balancer_arn = aws_lb.app.arn
  port              = 80
  protocol          = "HTTP"

  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.app.arn
  }
}
```

terraform/ecs.tf

```
resource "aws_ecs_cluster" "main" {
  name = "${var.project_name}-cluster"
}

resource "aws_cloudwatch_log_group" "app" {
  name              = "/ecs/${var.project_name}"
  retention_in_days = 7
}
```

```
resource "aws_ecs_task_definition" "app" {
  family                   = var.project_name
  requires_compatibilities = ["FARGATE"]
  network_mode             = "awsvpc"
  cpu                      = 256
  memory                   = 512
  execution_role_arn       = aws_iam_role.ecs_task_execution_role.arn
  task_role_arn             = aws_iam_role.ecs_task_role.arn

  container_definitions = jsonencode([
    {
      name      = "app"
      image     = "${aws_ecr_repository.app.repository_url}:latest"
      essential = true
      portMappings = [
        {
          containerPort = var.container_port
          protocol      = "tcp"
        }
      ]
      environment = [
        { name = "APP_VERSION", value = var.app_version },
        # TODO(candidate): plaintext secret in a task definition. This is
        # the one you're fixing for real - see Part 2, item 1.
        { name = "DB_PASSWORD", value = var.db_password }
      ]
      logConfiguration = {
        logDriver = "awslogs"
        options = {
          "awslogs-group"         = aws_cloudwatch_log_group.app.name
          "awslogs-region"        = var.aws_region
          "awslogs-stream-prefix" = "app"
        }
      }
    }
  ])
}

resource "aws_ecs_service" "app" {
  name            = "${var.project_name}-service"
  cluster         = aws_ecs_cluster.main.id
  task_definition = aws_ecs_task_definition.app.arn
  desired_count   = 1
  launch_type     = "FARGATE"

  # NOTE: tuned down from the default so deploys "feel" faster in testing.
  health_check_grace_period_seconds = 5

  deployment_circuit_breaker {
    enable   = true
    rollback = true
  }

  network_configuration {
    subnets          = data.aws_subnets.default.ids
    security_groups  = [aws_security_group.ecs_tasks.id]
    assign_public_ip = false
  }

  load_balancer {
    target_group_arn = aws_lb_target_group.app.arn
    container_name    = "app"
    container_port    = var.container_port
  }
```

```
  depends_on = [aws_lb_listener.app]
}
```

terraform/outputs.tf

```
output "alb_dns_name" {
  value = aws_lb.app.dns_name
}

output "ecr_repository_url" {
  value = aws_ecr_repository.app.repository_url
}
```

.github/workflows/deploy.yml

```
name: deploy

on:
  push:
    branches: [main]

env:
  AWS_REGION: us-east-1
  ECR_REPOSITORY: devops-assignment

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: ${{ env.AWS_REGION }}

      - name: Terraform apply
        working-directory: terraform
        run: terraform apply -auto-approve

      - name: Terraform init
        working-directory: terraform
        run: terraform init

      - name: Login to ECR
        run: |
          docker login --username AWS ${{ env.ECR_REGISTRY }}

      - name: Build and push image
        run: |
          docker build -t $ECR_REPOSITORY:latest ./app
          docker push $ECR_REPOSITORY:latest

      - name: Force new deployment
        run: |
          aws ecs update-service \
            --cluster devops-assignment-cluster \
            --service devops-assignment-service \
            --force-new-deployment
```